

Layovered — Referral App (Mobile): Step-by-Step Implementation Guide

Who this is for: the app developer (iOS + Android) building the referral experience, written so you can follow it even if you've never built a referral feature before. It tells you each screen to build, which API it calls, what to show in every state, when to fire analytics, and how to keep the copy correct.

0. The whole app job in one paragraph (read this first)

The backend does all the reward math. **Your job is the experience:** (1) an **Invite screen** where a user sees their link and shares it, (2) **attribution** — when a friend installs via the link, tell the backend who referred them, (3) a **search counter** on the search page that shows how many searches are left and nudges people to invite when they run low, (4) the **"out of searches" wall** — the single best place to ask for an invite, and (5) **confirmations** so both people see the reward land. The app never decides who gets a reward — it just shows what the backend says.

Two rules you must follow the whole way through:

1. **Never hardcode the numbers.** "+5", "+3", "/10" all come from the backend's `rewardConfig` (returned by `GET /referral/me`). Template every string from it. (So if marketing later switches the reward to a ₹500 coupon, the app shows the new thing with no update.)
 2. **The app never grants rewards.** It only reads state from the backend and displays it.
-

1. The 3 APIs you'll call (what each gives you)

You only talk to three backend endpoints. Here's the short version of what you do with each.

1.1 `GET /referral/me` — to fill the Invite screen

Returns the user's referral code, ready-made link, current bonus balance, how many rewards they've earned this month, the monthly cap, and the **reward config** (the numbers for your copy).

```
{
  "referralCode": "a7k9m2qx",
  "referralLink": "https://layovered.chottu.link/get-app?referrerId=a7k9m2qx",
  "bonusBalance": 8,
  "rewardedThisMonth": 3,
  "monthlyCap": 10,
  "rewardConfig": { "referrerReward": {"type":"searches","value":5},
                    "refereeReward": {"type":"searches","value":3} }
}
```

Use `referralLink` for sharing, `bonusBalance/rewardedThisMonth/monthlyCap` for the balance row, and `rewardConfig` for every "+5"/"+3" in your copy.

1.2 `POST /referral/attribute` — to credit the referrer at install

You send the `referrerId` you read from the deep link, plus the device id:

```
{ "referrerCode": "a7k9m2qx", "deviceId": "device-abc-123",
  "refereeUserId": 5567 }
```

(`refereeUserId` only if the new user is already signed in — otherwise leave it out.)

1.3 `GET /searches/remaining` — to drive the counter and its states

```
{ "dailyRemaining": 1, "bonusBalance": 8, "totalRemaining": 9 }
```

`totalRemaining` is the number you show on the chip. When it's 0, show the out-of-searches wall.

2. The screens & states you'll build (and the Figma for each)

Everything below is already designed in Figma (file "Layovered - ORIGINAL"). Build to match.

What	Figma screen	Driven by
Invite screen (referral hub)	"Referral – Invite friends" / "(402)"	<code>GET /referral/me</code>
Profile "Invite friends" row	row added on the profile/settings screen	navigation only
Search counter chip + states	search screen with "... searches left for today"	<code>GET /searches/remaining</code>
Out-of-searches wall	"Referral – Out of searches (sheet)"	counter == 0
"How searches work" explainer	"Referral – Know more (sheet)"	tap the chip

3. The Invite screen — step by step

- 1. On open, call `GET /referral/me`.
- 2. Fill the screen (strings in `REFERRAL_COPY.md` §1):
 - **Title / subtitle** — subtitle includes the reward numbers → pull from `rewardConfig` (e.g. "they get +3 to start, and you get +5 every time a friend runs their first search").
 - **Balance row** — "Searches earned: {{bonusBalance}} · Rewards left this month: {{monthlyCap – rewardedThisMonth}}/{{monthlyCap}}".
 - **Primary CTA** — "Share your invite link" → opens the share flow (Section 4).

- **Secondary** — "Copy link" (copies `referralLink`) and "Show QR" (render a QR of `referralLink`).
 - **Fine print** — "Rewards apply once your friend signs in with Google/Apple and runs their first search."
3. **Cap state:** if `rewardedThisMonth >= monthlyCap`, swap the messaging to the "cap reached" copy (`REFERRAL_COPY.md` §7) — but keep the link working.

Profile entry point: the "Invite friends" row on the profile/settings screen just navigates here.

4. Sharing the link — step by step

1. When the user taps **Share**, get the referral link:
 - Simplest: use `referralLink` from `/referral/me` (the backend already put `referrerId` in it).
 - Or build it via the ChottuLink SDK with `referrerId = referralCode` and `~feature=referral` (see `DEEPLINK_SETUP_BRIEF.md` §3.3).
2. Open the **native share sheet** with the **prefilled message** from `REFERRAL_COPY.md` §2 — the *referrer's own voice* ("I've been using Layovered..."), link auto-appended. Use the shorter Stories variant where space is tight.
3. **Fire the `invite_sent` analytics event.**

Why the referrer-voice message matters: a personal "I've been using this" converts far better than a branded "Check out Layovered!". Keep it personal.

5. Attribution — telling the backend who referred a new user

This is the part that makes the reward possible. It rides on the deep-link work (tickets LAY-2/LAY-3).

1. On the **first open after a fresh install**, read `referrerId` from the **deferred deep link** (the deep-link routing from LAY-3 hands you the params).
2. Call `POST /referral/attribute` with `{ referrerCode: referrerId, deviceId, refereeUserId? }`.
 - Include `refereeUserId` only if the user is already signed in; otherwise leave it out and the backend keeps the referral "pending".
3. **Call it once per install.** Set a local flag after the first successful call so you don't re-send it on every app open.
4. If the user **signs in later**, re-call `attribute` (now with `refereeUserId`) so the pending referral gets linked to their account.
5. **Fire `referral_install` analytics** with the `referrerId / ~channel`.

What you do **not** do: you don't check anti-abuse, you don't grant anything. The backend validates at the friend's first search. You just report the install.

6. The search counter and its states (the engine of the loop)

On the Layovered-mode search screen, call `GET /searches/remaining` and show state based on `totalRemaining`. Strings in `REFERRAL_COPY.md` §8.

State	Condition	What to show
Normal	<code>totalRemaining >= 2</code>	Chip near the toggle: "{{totalRemaining}} searches left today". Tappable → explainer sheet.
Low	<code>totalRemaining == 1</code>	Chip: "1 search left today · invite a friend for + {{value}} →". Tap → Invite screen.
Returning / how-to-get-more	screen opens with <code>totalRemaining</code> 0 or low	Slim banner: "Out of searches? Here's how to get more →" → Invite screen.
Exhausted	<code>totalRemaining == 0</code> and user tries to search	Open the out-of-searches wall (Section 7). Never show a dead error.

Refresh `totalRemaining` after every search and when the screen regains focus.

The numbers come from the API (`totalRemaining`) and the "+5" nudge from `rewardConfig` — nothing hardcoded.

7. The out-of-searches wall (your most important screen)

When a user with `totalRemaining == 0` tries to search, show the wall (Figma: "Out of searches (sheet)", copy: [REFERRAL_COPY.md](#) §3):

- Title: "You're out of searches for today".
- Body: the "+5 for you / +3 for them" offer (from `rewardConfig`).
- Primary CTA: "Invite a friend → +5 searches" → the share flow (Section 4).
- Secondary: "Or come back tomorrow for 2 more."

This is the single highest-converting moment in the whole loop — the user wants more searches *right now*, which is exactly when inviting is most appealing. Give it the polish the Figma shows.

8. Confirmations & welcome (so the reward feels real)

Strings in [REFERRAL_COPY.md](#) §5–6.

- **Referrer confirmation:** when a reward lands (the friend ran their first search), show a toast: 🎉
"{{friend_name}} just joined and ran their first search — you earned +{{value}} searches!" How you detect it: either refresh `/referral/me` and notice the balance went up, or use a push notification if you add one. Simplest first version: refresh `/referral/me` on app foreground and toast if the balance increased since last seen.
- **Referee welcome (the new friend):**
 - On first open via a link: "Welcome! {{friend_name}} invited you — run your first routing search to unlock +{{value}} bonus searches."
 - After their first search: "+{{value}} searches added. Happy hunting ➡".

9. Edge states — what to show when

Situation	What the app shows
Referrer hit the monthly cap	Invite screen "cap reached" copy; link still works (§7 copy).
Friend installed but hasn't activated	Optional "pending" line: "{{friend_name}} joined — they need to run their first search for your +{{value}} to land."
Friend skipped login (anonymous)	Messaging that they must sign in with Google/Apple for the reward to count.
<code>/referral/me</code> or <code>/searches/remaining</code> fails	Fail gracefully — show the screen without the dynamic numbers rather than blocking; retry on next focus.

10. Analytics — fire these two (the rest are backend)

- `invite_sent` — when the user shares (Section 4).
- `referral_install` — on the deferred open that carried a `referrerId` (Section 5).

The backend fires `referral_activated` and `reward_granted`. Together these let us compute the viral coefficient **K** and the invite→activation funnel (REFERRAL_LOOP_SPEC.md §9). Make sure your two events carry `referrerId` / `~channel` so they join up with the backend's.

11. How to test it (do these before "done")

1. **Invite screen** loads, shows link + QR + correct balance/cap, all numbers match config.
2. **Share** opens the share sheet with the referrer-voice message + link; `invite_sent` fires once.
3. **Fresh install via a referral link** → on first open, `POST /referral/attribute` fires once with the right `referrerId` + `deviceId`; `referral_install` fires. Reopening the app does **not** re-fire.
4. **Anonymous** → **sign-in later** → attribution links to the account after sign-in.
5. **Counter** shows the right `totalRemaining`; updates after a search.
6. **Low state** at exactly 1 remaining shows the invite nudge.
7. **Exhausted** (0) → searching opens the wall, not an error; wall CTA opens the share flow.
8. **Explainer sheet** opens from the chip; CTA opens Invite.
9. **Referrer confirmation** appears after a test friend activates; **referee welcome** appears before and after the friend's first search.
10. **Config swap test**: change the reward value server-side → every "+5/+3" in the app updates with no app rebuild. (This proves rule #1.)

12. Build order (what to do first)

1. **LAY-REFAPP-1** — wire `GET /searches/remaining` into the search screen.
2. **LAY-REFAPP-5, 6, 7** — counter states, out-of-searches wall, explainer. (These only need the searches API, so you can ship visible value before the rest of the backend/deep-link is ready.)
3. **LAY-REFAPP-2, 3** — Invite screen + share (needs `/referral/me` + ChottuLink SDK / LAY-2).
4. **LAY-REFAPP-4** — attribution (needs deep-link routing LAY-3 + `/referral/attribute`).
5. **LAY-REFAPP-8, 9, 10** — confirmations, edge states, analytics.